RTXPT Python API Reference

This document describes how to use the current `rtxpt` Python bindings. The API is primarily defined in:

- `Rtxpt/Python/PythonBindingsCore.cpp`
- `Rtxpt/Python/PythonBindings_Extension.cpp`
- `Rtxpt/Python/PythonBindings_Embed.cpp`
- `Rtxpt/Python/RenderSession.*`

Table of Contents

Two Usage Modes

The `rtxpt` module supports two runtime modes that share most types:

Mode	How to use	Typical use
`extension`	`import rtxpt` in a standalone Python process and create `rtxpt.Renderer(...)`	Offline rendering, batch jobs, screenshots, automated tests, quick 3DGS validation
`embed`	`import rtxpt` from the in-app script system inside a running `Rtxpt.exe`	Live parameter tuning, debugging, hot edits to scenes/materials/lights

At runtime you can check the active mode with:

```
import rtxpt
print(rtxpt.MODE)  # "extension" or "embed"
```

In extension mode, `rtxpt.Renderer` creates its own window, device, and scene.
In embed mode there is no `Renderer` class; use `rtxpt.app()` to access the renderer inside the running `Rtxpt.exe`.

Import Setup

After building the `rtxpt_py` target, the Python extension is emitted under `bin/`:

```
bin/rtxpt.cp311-win_amd64.pyd
```

The recommended install path is to run from the repository root:

```
python -m pip install .
python -c "import rtxpt; print(rtxpt.MODE)"
```

This assembles a local binary wheel from the native extension, runtime DLLs/shared libraries, shaders, and required assets in `bin/`, then installs it into the active Python environment. You can also build the wheel explicitly first:

```
python Support/python/build_wheel.py
python -m pip install dist/rtxpt-*.whl
```

Packaging options can be controlled with environment variables:

Variable	Default	Values
`RTXPT_WHEEL_VERSION`	`0.2.0`	Any PEP 440 version string
`RTXPT_WHEEL_ASSETS`	`minimal`	`minimal`, `full`, `none`
`RTXPT_WHEEL_DYNAMIC_SHADERS`	`bin`	`bin`, `full`, `none`
`RTXPT_WHEEL_SHADER_API`	`d3d12` on Windows, `vulkan` elsewhere	`d3d12`, `vulkan`, `both`

During development, if you prefer not to install the package, you can still add `bin/` to `sys.path` or `PYTHONPATH`:

```
import sys
sys.path.insert(0, r"D:\ProgramCode\C++\RTXPT\bin")

import rtxpt
```

See `configure_import_path()` in `Rtxpt/Python/Examples/test_splat_interactive.py` for another example.

Quick Examples

Headless Reference Render

```
import rtxpt

with rtxpt.Renderer(
    width=1280,
    height=720,
    headless=True,
    scene="bistro-programmer-art.scene.json",
```

```
    realtime=False,
    accumulation_target=64,
) as r:
    r.settings.enable_tone_mapping = True
    frames = r.step_until_accumulated()
    print("frames:", frames)
    r.save_screenshot("frame.png")
```

Windowed Interactive Loop

```
import time
import rtxpt

r = rtxpt.Renderer(
    width=1280,
    height=720,
    headless=False,
    scene="bistro-programmer-art.scene.json",
    realtime=True,
    accumulation_target=1,
)

try:
    while r.step(-1.0):  # returns False if the window is closed
        time.sleep(0.001)
finally:
    r.close()
```

Load 3D Gaussian Splats

3DGS objects are scene graph objects. Prefer declaring them in the scene JSON:

```
import rtxpt

scene = r'''
{
  "models": ["builtin:plane"],
  "graph": [
    { "name": "Ground", "model": 0 },
    {
      "name": "Scan",
      "type": "GaussianSplat",
      "path": "D:/ScanVideo/chuan/splats.ply",
      "convertRdfToDonut": true,
      "translation": [0, 0, 0],
      "scaling": [1, 1, 1]
    }
  ]
}
'''

r = rtxpt.Renderer(width=1280, height=720, headless=False, realtime=True, scene=scene)

s = r.settings
s.enable_gaussian_splats = True
s.gaussian_splat_sorting_mode = int(rtxpt.GaussianSplatSortMode.GpuSort)
s.gaussian_splat_sh_format = int(rtxpt.GaussianSplatStorageFormat.Uint8)
```

```
s.gaussian_splat_rgba_format = int(rtxpt.GaussianSplatStorageFormat.Uint8)
s.gaussian_splat_scale = 1.0
s.gaussian_splat_alpha_scale = 1.0
s.gaussian_splat_brightness = 1.0

while r.step(-1.0):
    pass
```

For script-driven workflows, `load_gaussian_splats(path, convert_rdf_to_donut=True)` appends a `GaussianSplat` node to the current scene root. Calling `load_scene(...)` replaces the current scene graph and destroys previously appended splat nodes, so load them again after switching scenes or declare them in the target scene JSON.

3DGS Reference / Realtime Batch Test

`3dgs_example.py` renders the same PLY twice:

- Reference mode accumulates 32 spp, then applies OIDN and writes `reference_oidn.png`.
- Realtime mode steps 32 frames and uses DLSS-RR when supported, falling back to DLSS/TAA/off, then writes `realtime_<aa>.png`.
- The default 3DGS sorting mode is GPU sort. Pass `--sorting stochastic` to compare with stochastic splats.

```
python .\Rtxpt\Python\Examples\3dgs_example.py ^
    --ply D:/ScanVideo/chuan/splats.ply ^
    --out-dir 3dgs_chuan_gpu_sort_out
```

Useful camera overrides:

```
python .\Rtxpt\Python\Examples\3dgs_example.py ^
    --ply D:/ScanVideo/chuan/splats.ply ^
    --out-dir 3dgs_chuan_out ^
    --distance-scale 4.0 ^
    --side front
```

COLMAP Camera 3DGS Alignment Test

`render_gs_colmap_views.py` renders a 3DGS PLY from COLMAP `cameras.bin/images.bin` views. It is useful for comparing RTXPT output against gsplat output from the same camera poses.

By default, it reads:

```
D:/ProgramCode/Python/demo_gsplat&blender/GS/gaussians.ply
D:/ProgramCode/Python/demo_gsplat&blender/GS/sparse
```

Example:

```
python .\Rtxpt\Python\Examples\render_gs_colmap_views.py ^
    --max-views 8 ^
    --frames-per-view 8 ^
    --warmup-frames 4 ^
    --mip-antialiasing ^
    --out-dir "D:\ProgramCode\Python\demo_gsplat&blender\GS\rtxpt_rendered_intrinsics_mipaa"
```

The script passes full COLMAP pinhole intrinsics (`fx`, `fy`, `cx`, `cy`) through `Renderer.set_camera_intrinsics(...)`. This keeps off-center principal points aligned with gsplat. Use `--symmetric-fov` only when intentionally testing the older vertical-FOV-only path.

When `--convert-rdf-to-donut` is enabled, which is the default, both the PLY loader and the COLMAP camera pose are converted from RDF/COLMAP coordinates into RTXPT/Donut coordinates. `--mip-antialiasing` is enabled by default and can be disabled with `--no-mip-antialiasing`.

Load OBJ Meshes With Materials

`Renderer.load_mesh_file(...)` and `Sample.load_mesh_file(...)` append OBJ models to the current scene. The loader parses `mtllib` directives in the OBJ file and resolves `.mtl` and texture paths relative to the OBJ/MTL directory by default.

The current OBJ/MTL importer recognizes these common material fields:

- Scalars/colors: `Kd`, `Ks`, `Ke`, `Ns`, `Pr`, `Pm`, `Ni`, `d`, `Tr`, `Tf`.
- Base color / diffuse maps: `map_Kd`, `map_basecolor`.
- PBR metal-roughness: `map_Pr`, `map_roughness`, `map_Pm`, `map_metallic`, `map_metalness`.
- Packed PBR maps: `map_orm`, `map_mr`, `map_metallicroughness`, `map_occlusionroughnessmetallic`.
- AO / occlusion: `map_Ka`, `map_ao`, `map_occlusion`.
- Normal / bump: `map_Bump`, `bump`, `norm`, `map_normal`, including `-bm` strength.
- Specular / glossiness: `map_Ks`, `map_Ns` for specular-gloss materials.
- Emissive / opacity / transmission: `map_Ke`, `map_emissive`, `map_d`, `map_opacity`, `map_Tf`.

When an MTL file provides separate roughness and metallic maps, the importer builds an in-memory ORM texture for RTXPT: `R=AO`, `G=roughness`, `B=metallic`, `A=1`. The `-imfchan` channel selector is honored when reading single-channel maps. If `map_Ke` or `map_emissive` is present without an explicit `Ke` color, the importer uses `(1, 1, 1)` as the emissive factor so the emissive texture is visible.

```
import rtxpt

obj_path = r"D:/assets/m-plate-pbr_final/textured.obj"

with rtxpt.Renderer(scene="builtin:plane", headless=True, accumulation_target=32) as r:
    if not r.load_mesh_file(obj_path):
        raise RuntimeError(f"failed to load {obj_path}")

    # Imported materials are available after the mesh is appended and at least
    # one update frame has run.
    r.step_n(1)

    for mat in r.app.scene.get_materials():
        print(mat.model_name, mat.name, mat.base_color, mat.roughness, mat.metalness)

    r.step_until_accumulated()
    r.save_screenshot("obj_materials.png")
```

Edit Materials

```
import rtxpt

with rtxpt.Renderer(scene="bistro-programmer-art.scene.json", headless=True) as r:
    scene = r.app.scene

    # Names can come from scene.get_materials(), the MTL `newmtl` name, or the
    # material unique_name printed below.
```

```
    for mat in scene.get_materials():
        print(mat.model_name, mat.name, mat.unique_name)

    mat = scene.find_material("SomeMaterialName")
    if mat is None:
        raise RuntimeError("material not found")

    # Scalars/colors multiply the loaded texture values when the corresponding
    # texture remains enabled.
    mat.base_color = (1.0, 0.2, 0.1)
    mat.roughness = 0.35
    mat.metalness = 0.0
    mat.normal_texture_scale = 0.75

    # Texture bindings can be enabled/disabled, or replaced from Python.
    mat.enable_base_texture = False
    mat.enable_orm_texture = True
    mat.enable_normal_texture = True
    mat.set_base_texture(r"D:/assets/replacement_albedo.png")
    mat.set_normal_texture(r"D:/assets/replacement_normal.png")

    # In reference accumulation mode, reset after any visible edit so old
    # accumulated samples do not remain mixed into the image.
    r.app.reset_accumulation()

    r.step_n(4)
    r.save_screenshot("material_edit.png")
```

All writable `Material` properties mark the material GPU data dirty automatically; calling `mark_dirty()` is only needed if native-side data was changed without going through a Python property setter. Texture replacement helpers load the new image through the runtime texture cache, enable the slot, and upload updated material data on the next rendered frame.

Read and Replace Material Textures

Texture access is exposed as file bindings on `Material`. Python can read the current loaded texture path, replace a slot with another image file, enable or disable an existing slot, and clear a slot. It does not expose direct pixel-buffer editing.

```
import rtxpt

with rtxpt.Renderer(scene="bistro-programmer-art.scene.json", headless=True) as r:
    mat = r.app.scene.find_material("SomeMaterialName")
    if mat is None:
        raise RuntimeError("material not found")

    # Read current texture file bindings. Each property is str or None.
    print("base:", mat.base_texture_path)
    print("orm:", mat.orm_texture_path)
    print("normal:", mat.normal_texture_path)
    print("emissive:", mat.emissive_texture_path)
    print("transmission:", mat.transmission_texture_path)

    # Replace common slots with slot-specific helpers.
    if not mat.set_base_texture(r"D:/assets/albedo_replacement.png"):
        raise RuntimeError("base texture not found")
    mat.set_normal_texture(r"D:/assets/normal_replacement.png")

    # Replace any slot with the generic API and TextureSlot enum.
    mat.set_texture(rtxpt.TextureSlot.Emissive, r"D:/assets/emissive.png")
    mat.set_texture(rtxpt.TextureSlot.ORM, r"D:/assets/orm_linear.png", srgb=False)
```

```
    # Toggle sampling without disconnecting the loaded texture.
    mat.enable_base_texture = True
    mat.enable_normal_texture = False

    # Disconnect and disable a slot.
    mat.clear_emissive_texture()
    # Equivalent generic form:
    mat.clear_texture(rtxpt.TextureSlot.Transmission)

    r.app.reset_accumulation()
    r.step_n(4)
```

`set_*_texture(...)` returns `True` when the file was resolved and loaded, `False` when it was not found. A successful set operation also enables that texture slot. `clear_*_texture(...)` removes the binding and disables the slot.

Default color-space handling:

Slot	Helper	Default interpretation
`TextureSlot.Base`	`set_base_texture(path, srgb=None)`	sRGB
`TextureSlot.ORM`	`set_orm_texture(path, srgb=None)`	Linear in metal-rough mode, sRGB in spec-gloss mode
`TextureSlot.Normal`	`set_normal_texture(path)`	Linear normal map
`TextureSlot.Emissive`	`set_emissive_texture(path, srgb=None)`	sRGB
`TextureSlot.Transmission`	`set_transmission_texture(path, srgb=None)`	Linear

Use the optional `srgb` argument to override the default color-space choice. The generic `set_texture(slot, path, srgb=None, normal_map=None)` also accepts `normal_map` for advanced cases; leave it as `None` for the slot default.

Relative texture paths are resolved through the same runtime texture search used by material JSON loading: runtime `Assets/` first, then the current scene directory. For `.png` inputs, an existing sibling `.dds` is preferred, matching RTXPT material loading behavior.

Edit Lights

```
import rtxpt

r = rtxpt.Renderer(scene="bistro-programmer-art.scene.json", headless=True)
scene = r.app.scene
for light in scene.get_lights():
    print(light.name, light.light_type)
    light.color = (1.0, 0.9, 0.75)

sun = scene.find_light("Sun")
if sun:
    sun.direction = (0.0, -1.0, 0.2)

r.step_n(8)
r.save_screenshot("lights.png")
r.close()
```

Deform Mesh Vertices

Mesh deformation works on object-space vertex positions. After `set_mesh_vertices(...)` or `deform_mesh(...)`, RTXPT refreshes the mesh GPU buffer and can rebuild ray tracing acceleration structures so the edited geometry is used by subsequent frames.

```
import math
import rtxpt

r = rtxpt.Renderer(scene="builtin:cube", headless=True, accumulation_target=8)
app = r.app

mesh = app.find_mesh("cube") or app.get_meshes()[0]
vertices = list(app.get_mesh_vertices(mesh))

# Simple soft bulge: move upper vertices upward based on x/z radius.
deformed = []
for x, y, z in vertices:
    radius = math.sqrt(x * x + z * z)
    lift = 0.15 * max(0.0, 1.0 - radius)
    deformed.append((x, y + lift, z))

app.set_mesh_vertices(mesh, deformed, recompute_normals=True)
app.step_until_accumulated()
r.save_screenshot("deformed_mesh.png")
r.close()
```

For callback-style edits, return `None` to keep a vertex unchanged:

```
def wave(index, p):
    x, y, z = p
    if y < 0:
        return None
    return (x, y + 0.05 * math.sin(index * 0.37), z)

app.deform_mesh(mesh, wave, recompute_normals=True)
```

Use the `_world` variants when the values you read and return should be scene world coordinates. Passing a `Mesh` works when that mesh has exactly one scene instance. For instanced/shared meshes, pass the owning `SceneNode` so RTXPT can use that node's local-to-world transform:

```
node = app.find_node("cube")  # or any mesh node/path

def lift_world(index, p):
    x, y, z = p
    return (x, y + 0.25, z)

app.deform_mesh_world(node, lift_world, recompute_normals=True)
```

API Reference

The sections below are grouped by topic so you can jump directly to the API you need:

Category	Section
Renderer	Renderer

Category	Section
Scene / app	Sample & Scene
Mesh / nodes	Model
Materials	Material
Lights	Light
Gaussian splats	3DGS
Render settings	Settings
Shared enums	Enums

Module-Level API

These functions exist in both embed and extension mode unless noted.

API	Return	Notes
`rtxpt.MODE`	`str`	`"embed"` or `"extension"`.
`rtxpt.app()`	`Sample`	Current renderer. In extension mode, returns the most recently created `Renderer`'s `Sample`.
`rtxpt.settings()`	`Settings`	Shortcut to global live UI/settings state. Same object as `rtxpt.app().settings`.
`rtxpt.log_info(message)`	`None`	Writes to RTXPT log at info level.
`rtxpt.log_warning(message)`	`None`	Writes to RTXPT log at warning level.
`rtxpt.log_error(message)`	`None`	Writes to RTXPT log at error level.
`rtxpt.Renderer(...)`	`Renderer`	Extension mode only. Creates a standalone renderer/device/window or headless backbuffer.
`rtxpt.builtin_scene_json(builtin_model="plane_cube")`	`str`	Extension mode only. Returns minimal inline scene JSON for `plane`, `cube`, `sphere`, or `plane_cube`.

Renderer

Extension mode only. Create with `rtxpt.Renderer(...)`.

Constructor

```
rtxpt.Renderer(
    width=1920,
    height=1080,
    headless=True,
    vulkan=False,
    adapter_index=-1,
    debug=False,
    scene="",
    realtime=False,
    accumulation_target=64,
)
```

Argument	Meaning
`width`, `height`	Initial backbuffer/window size.
`headless`	`True`: offscreen backbuffers, no OS window. `False`: create a window and swap chain.
`vulkan`	`False` uses DX12. `True` requests Vulkan when available.
`adapter_index`	GPU index, `-1` means default adapter.
`debug`	Enable graphics debug settings.
`scene`	Scene file path/name, `builtin:*` primitive reference, or inline scene JSON string. Relative file paths are resolved from `Assets/`.
`realtime`	Start in realtime mode if `True`, reference mode if `False`.
`accumulation_target`	Reference SPP target.

Methods / Properties

API	Return	Notes
`close()`	`None`	Tears down renderer/device. Also called by destructor/context manager.
`load_scene(scene_name, wait_until_ready=True)`	`bool`	Switch scene.
`load_gaussian_splats(file_name, convert_rdf_to_donut=True)`	`bool`	Append a `.ply` 3DGS scene object under the current scene root.
`load_mesh_file(file_name)`	`bool`	Append a `.gltf`, `.glb`, or `.obj` mesh under the current scene root. OBJ imports resolve referenced `.mtl` files and common material textures relative to the OBJ/MTL path.
`get_scene_bounds()`	`tuple | None`	Active scene world-space `((min.xyz), (max.xyz))` AABB from C++ `Scene::GetSceneBounds()`.
`scene_bounds`	`tuple | None`	Property alias for `get_scene_bounds()`.
`scene_bounds_center`	`tuple | None`	Center of `scene_bounds`.
`scene_bounds_size`	`tuple | None`	Extent `(max - min)` of `scene_bounds`.
`step(dt=-1.0)`	`bool`	Render one frame. Returns `False` on failure or when window close is requested.
`step_n(frames)`	`bool`	Render exactly N frames unless `step()` fails.
`step_until_accumulated(max_frames=0)`	`int`	Reset accumulation and step until accumulation completes, or until `max_frames` if positive.
`save_screenshot(output_path)`	`bool`	Save current backbuffer to PNG/JPG/BMP/TGA.
`set_camera(position, direction, up=(0, 1, 0))`	`bool`	Triples can be lists/tuples of 3 floats.
`set_camera_fov(vertical_fov_degrees)`	`None`	Set vertical FOV in degrees.

API	Return	Notes
`set_camera_intrinsics(fx, fy, cx, cy, width, height)`	`None`	Set an off-center pinhole projection from pixel-space intrinsics. This overrides the symmetric FOV projection until `set_camera_fov(...)` is called.
`app`	`Sample`	Underlying renderer instance.
`settings`	`Settings`	Live UI/settings state.

`Renderer` supports context manager syntax:

```
with rtxpt.Renderer(headless=True) as r:
    r.step_n(8)
```

Inline / Builtin Scenes

For package smoke tests that should not depend on external mesh assets, the extension accepts builtin primitive scenes:

```
with rtxpt.Renderer(headless=True, scene="builtin:plane_cube", accumulation_target=4) as r:
    r.step_until_accumulated()
    r.save_screenshot("smoke.png")
```

Supported builtin models are `builtin:plane`, `builtin:cube`, `builtin:sphere`, and `builtin:plane_cube`.

You can also pass an inline scene JSON string. Model entries may reference builtin primitives:

```
scene = rtxpt.builtin_scene_json("plane_cube")
r = rtxpt.Renderer(headless=True, scene=scene)
```

Scene JSON may also declare one or more 3DGS nodes directly:

```
scene = r"""
{
  "graph": [
    {
      "name": "ScanA",
      "type": "GaussianSplat",
      "path": "D:/ScanVideo/chuan/splats_a.ply",
      "translation": [0.0, 0.0, 0.0],
      "scaling": 1.0,
      "convertRdfToDonut": true,
      "enabled": true
    },
    {
      "name": "ScanB",
      "type": "GaussianSplat",
      "path": "D:/ScanVideo/chuan/splats_b.ply",
      "translation": [2.0, 0.0, 0.0],
      "scaling": 0.75
    }
  ]
}
```

```
"""
r = rtxpt.Renderer(headless=False, realtime=True, scene=scene)
```

For scene files, relative 3DGS paths are resolved relative to the scene JSON file. `path`, `file`, and `fileName` are accepted aliases.

Sample & Scene

Top-level renderer instance (`Sample`). In extension mode, access it through `renderer.app`; in embed mode, use `rtxpt.app()`. Scene graph access goes through `app.scene`.

Read-Only Properties

Property	Type	Notes
`settings`	`Settings`	Live settings object.
`scene`	`Scene | None`	Current loaded scene, matching the C++ `GetScene()` entry point.
`scene_name`	`str`	Current scene name.
`available_scenes`	`list[str]`	Scene files discovered by the app.
`gaussian_splat_object_count`	`int`	Number of loaded 3DGS scene objects.
`gaussian_splat_count`	`int`	Total loaded splat count across current 3DGS scene objects.
`gaussian_splat_file_name`	`str`	Single loaded 3DGS path, or a summary when multiple 3DGS objects are present.
`scene_bounds`	`tuple | None`	Shortcut for `scene.get_scene_bounds()`.
`scene_bounds_center`	`tuple | None`	Center of `scene_bounds`.
`scene_bounds_size`	`tuple | None`	Extent `(max - min)` of `scene_bounds`.
`accumulation_completed`	`bool`	Whether reference accumulation is complete.
`accumulation_sample_index`	`int`	Current accumulation sample index.

Scene / Assets

API	Return	Notes
`set_scene(scene_name, force_reload=False)`	`None`	Switch scene.
`load_gaussian_splats(file_name, convert_rdf_to_donut=True)`	`bool`	Append a 3DGS `.ply` node to the current scene.
`load_mesh_file(file_name)`	`bool`	Append a `.gltf`, `.glb`, or `.obj` mesh node to the current scene. OBJ imports resolve referenced `.mtl` files and common material textures relative to the OBJ/MTL path.
`set_environment_map(path)`	`None`	Override scene environment map source.
`get_scene()`	`Scene | None`	Return the current loaded scene.

API	Return	Notes
`get_scene_bounds()`	`tuple | None`	Shortcut for `scene.get_scene_bounds()`.

Camera

API	Return	Notes
`get_camera_pos_dir_up()`	`str`	Comma-separated `pos.xyz,dir.xyz,up.xyz`.
`set_camera_pos_dir_up(pos_dir_up)`	`bool`	Input format matches `get_camera_pos_dir_up()`.
`set_camera_fov(vertical_fov_degrees)`	`None`	Takes degrees.
`set_camera_intrinsics(fx, fy, cx, cy, width, height)`	`None`	Uses pixel-space pinhole intrinsics for the active projection. Useful for COLMAP/OpenCV cameras with non-centered `cx/cy`.
`get_camera_fov()`	`float`	Returns current internal value in radians.
`save_current_camera()`	`None`	Save camera through app's camera persistence path.
`load_current_camera()`	`None`	Restore saved camera.

Use `Renderer.set_camera()` when working in extension mode; it is simpler than building the comma-separated string manually.

Runtime Requests

API	Effect
`request_shader_reload()`	Requests shader reload.
`request_accel_rebuild()`	Requests acceleration structure rebuild.
`reset_accumulation()`	Resets reference accumulation.

Mode Helpers

```
app.set_realtime_mode(
    standalone_denoiser=True,
    realtime_aa=int(rtxpt.RealtimeAA.DLSS),
)

app.set_reference_mode(
    spp=128,
    oidn=True,
    oidn_quality=int(rtxpt.OidnQuality.Balanced),
    oidn_passes=int(rtxpt.OidnPasses.Albedo),
    oidn_prefilter=int(rtxpt.OidnPrefilter.Fast),
)
```

API	Notes
`set_realtime_mode(standalone_denoiser=True, realtime_aa=2)`	Sets realtime mode. `realtime_aa`: `0=Off`, `1=TAA`, `2=DLSS`, `3=DLSS_RR`.
`set_reference_mode(spp=0, oidn=False, oidn_quality=1, oidn_passes=1, oidn_prefilter=1)`	Sets reference mode. `spp=0` keeps current target.

Model

Mesh geometry, scene graph nodes, and vertex deformation. Access meshes through `app.scene` or `Sample` compatibility aliases.

Scene Lookup

API	Return	Notes
`scene.get_meshes()`	`list[Mesh]`	All meshes in the current scene.
`scene.find_mesh(name)`	`Mesh | None`	Match by mesh name.
`scene.mesh_count`	`int`	Number of meshes in the current scene.
`scene.find_node(path)`	`SceneNode | None`	Find a scene graph node by name or path.
`sample.get_meshes()`	`list[Mesh]`	Compatibility alias for `scene.get_meshes()`.
`sample.find_mesh(name)`	`Mesh | None`	Compatibility alias for `scene.find_mesh(name)`.
`sample.find_node(path)`	`SceneNode | None`	Compatibility alias for `scene.find_node(path)`.
`Renderer.load_mesh_file(file_name)`	`bool`	Append a `.gltf`, `.glb`, or `.obj` mesh (extension mode).
`Sample.load_mesh_file(file_name)`	`bool`	Append a mesh node (embed or extension).

Vertex Deformation

API	Return	Notes
`sample.get_mesh_vertices(mesh)`	`list[tuple]`	Returns object-space `(x, y, z)` vertex positions.
`sample.set_mesh_vertices(mesh, vertices, recompute_normals=True, rebuild_acceleration_structure=True)`	`None`	Replaces all positions. `vertices` must contain exactly `mesh.vertex_count` triples.
`sample.deform_mesh(mesh, callback, recompute_normals=True, rebuild_acceleration_structure=True)`	`int`	Calls `callback(index, (x, y, z))` for each vertex. Return a new triple or `None`; returns the processed vertex count.
`sample.get_mesh_vertices_world(mesh_or_node)`	`list[tuple]`	Returns world-space `(x, y, z)` vertex positions. Pass a `SceneNode` for instanced/shared meshes.
`sample.set_mesh_vertices_world(mesh_or_node, vertices, recompute_normals=True, rebuild_acceleration_structure=True)`	`None`	Replaces all positions from world-space coordinates, converting through the selected node transform.
`sample.deform_mesh_world(mesh_or_node, callback, recompute_normals=True, rebuild_acceleration_structure=True)`	`int`	Callback receives world-space `(x, y, z)` and returns a world-space replacement or `None`.

`set_mesh_vertices(...)` updates object-space mesh bounds, optionally recomputes normals, refreshes GPU vertex data, resets accumulation, and requests acceleration structure rebuild by default. Keep `rebuild_acceleration_structure=True` for ray tracing-correct geometry. Only set it to `False` when batching several edits and calling `request_accel_rebuild()` after the final update.

The `_world` variants refresh the scene graph transform state before converting coordinates, so recent Python transform edits such as `node.translation = ...` are reflected immediately. The underlying mesh vertex buffer is still shared: editing a mesh through one node updates the mesh data used by any other nodes that instance the same `Mesh`.

Scene Bounds

API	Return	Notes
`scene.get_scene_bounds()`	`tuple | None`	World-space `((min.xyz), (max.xyz))` AABB from C++ `Scene::GetSceneBounds()`.
`scene.get_bounds()`	`tuple | None`	Alias for `scene.get_scene_bounds()`.
`scene.bounds`	`tuple | None`	Property alias for `scene.get_scene_bounds()`.
`scene.bounds_center`	`tuple | None`	Center of `scene.bounds`.
`scene.bounds_size`	`tuple | None`	Extent `(max - min)` of `scene.bounds`.
`Renderer.get_scene_bounds()` / `scene_bounds`	`tuple | None`	Extension-mode world bounds shortcut.

`Mesh` Class

Returned by `Scene.get_meshes()`, `Scene.find_mesh()`, `Sample.get_meshes()`, `Sample.find_mesh()`, and `SceneNode.mesh`.

Property	Type	Notes
`name`	`str`	Mesh name from the source model or builtin primitive.
`global_mesh_index`	`int`	Internal scene mesh index.
`vertex_count`	`int`	Number of object-space positions expected by `set_mesh_vertices(...)`.
`index_count`	`int`	Total index count.
`geometry_count`	`int`	Number of mesh geometry groups/submeshes.
`bounds`	`((min.xyz), (max.xyz)) | None`	Object-space mesh AABB.

Vertex data is edited through `Sample.set_mesh_vertices()` / `Sample.deform_mesh()`, not through writable `Mesh` properties, because changing vertices also needs GPU buffer refresh and acceleration-structure invalidation.

`SceneNode` Class

Returned by `Scene.find_node()` and `Sample.find_node()`.

Property	Type
`name`	`str`
`path`	`str`
`mesh`	`Mesh | None`
`is_mesh`	`bool`

Property	Type
`translation`	`(x, y, z)`
`rotation`	`(x, y, z, w)` quaternion
`euler`	`(x, y, z)` radians
`scaling`	`(x, y, z)`
`bounds`	`((min.xyz), (max.xyz)) | None`

`rotation` and `euler` both write the node's local Transform rotation. Assigning `euler` converts XYZ radians to the stored quaternion; assigning `rotation` expects an XYZW quaternion, matching scene JSON. Python Transform edits reset accumulation automatically so the next rendered frame does not blend with the previous pose.

Material

Scene material lookup and the `Material` class for runtime edits and texture replacement.

Scene Lookup

API	Return	Notes
`scene.get_materials()`	`list[Material]`	All `PTMaterial` materials in the current scene.
`scene.find_material(name)`	`Material | None`	Match by `Name` or `UniqueName`.
`scene.find_material_by_id(material_id)`	`Material | None`	Lookup by material ID.
`scene.material_count`	`int`	Number of PT materials in the current scene.

`Sample.get_materials()`, `Sample.find_material()`, and `Sample.find_material_by_id()` remain available as compatibility aliases.

`Material` Class

Returned by `Scene.get_materials()`, `Scene.find_material()`, and `Scene.find_material_by_id()`.

Identifiers (read-only)

Property	Type
`name`	`str`
`model_name`	`str`
`unique_name`	`str`

Properties (writable)

Editable properties automatically mark GPU data dirty:

Property	Type
`base_color`	`(r, g, b)`
`specular_color`	`(r, g, b)`
`emissive_color`	`(r, g, b)`
`emissive_intensity`	`float`

Property	Type
metalness	float
roughness	float
opacity	float
transmission_factor	float
diffuse_transmission_factor	float
normal_texture_scale	float
ior	float
alpha_cutoff	float
volume_attenuation_distance	float
volume_attenuation_color	(r, g, b)
nested_priority	int
use_specular_gloss	bool
enable_alpha_testing	bool
enable_transmission	bool
thin_surface	bool
exclude_from_nee	bool
enable_as_analytic_light_proxy	bool
skip_render	bool
metalness_in_red_channel	bool
enable_base_texture	bool
enable_orm_texture	bool
enable_normal_texture	bool
enable_emissive_texture	bool
enable_transmission_texture	bool

Texture Paths (read-only)

Property	Type
base_texture_path	str | None
orm_texture_path	str | None
normal_texture_path	str | None
emissive_texture_path	str | None
transmission_texture_path	str | None

Methods

API	Notes

API	Notes
`mark_dirty()`	Force material GPU buffer refresh next frame.
`set_texture(slot, path, srgb=None, normal_map=None)`	Replace a texture slot. `slot` is a `TextureSlot` enum value. Returns `False` if the file cannot be resolved.
`set_base_texture(path, srgb=None)`	Replace base/diffuse texture. Defaults to sRGB.
`set_orm_texture(path, srgb=None)`	Replace ORM/spec-gloss texture. Defaults to linear for metal-rough and sRGB for spec-gloss.
`set_normal_texture(path)`	Replace normal texture.
`set_emissive_texture(path, srgb=None)`	Replace emissive texture. Defaults to sRGB.
`set_transmission_texture(path, srgb=None)`	Replace transmission texture. Defaults to linear.
`clear_texture(slot)`	Disconnect and disable a texture slot.
`clear_base_texture()`, `clear_orm_texture()`, `clear_normal_texture()`, `clear_emissive_texture()`, `clear_transmission_texture()`	Slot-specific clear helpers.

Runtime Update Rules

- Property setters already mark `GPUDataDirty`; the edited material is uploaded on the next rendered frame.
- In reference/accumulation mode, call `Sample.reset_accumulation()` or set `settings.reset_accumulation = True` after visible edits, otherwise previous samples remain blended with the old material.
- Color values are linear RGB. The Python setter does not clamp inputs, so keep factors in the physically meaningful range unless deliberately testing extremes.
- If a texture slot is enabled, scalar/color parameters multiply the texture sample. In metal-rough mode, effective base color is `base_color * base_texture.rgb`, roughness is `roughness * ORM.g`, and metalness is `metalness * ORM.b` unless `metalness_in_red_channel=True`.
- `opacity` is multiplied by the base texture alpha when `enable_base_texture=True`.
- Use `set_base_texture`, `set_orm_texture`, `set_normal_texture`, `set_emissive_texture`, or `set_transmission_texture` to replace an imported texture at runtime. Relative paths are resolved the same way as material JSON paths: runtime `Assets/` first, then the current scene directory. For `.png` inputs, an existing sibling `.dds` is preferred, matching material JSON loading.
- Pure parameter edits such as color, roughness, metalness, opacity, texture toggles, emissive intensity, normal scale, and IOR are next-frame updates. Bigger classification edits such as `use_specular_gloss`, `enable_alpha_testing`, `alpha_cutoff`, `enable_transmission`, `exclude_from_nee`, or `skip_render` can change shader hit groups, alpha handling, lighting participation, or acceleration-structure metadata; after those edits, request a shader/acceleration refresh.

Typical runtime material override:

```
app = rtxpt.app()              # embed mode
# app = renderer.app           # extension mode
scene = app.scene

mat = scene.find_material("material_0")
if mat is not None:
    mat.base_color = (0.8, 0.9, 1.0)
    mat.roughness = 0.18
    mat.metalness = 0.85
    mat.set_base_texture(r"D:/assets/replacement_albedo.png")
    mat.enable_orm_texture = True
```

```
    mat.normal_texture_scale = 1.0
    app.reset_accumulation()
```

When changing material classification flags:

```
mat.enable_alpha_testing = True
mat.alpha_cutoff = 0.4
mat.enable_transmission = True
mat.transmission_factor = 0.6

app.reset_accumulation()
app.request_shader_reload()
app.request_accel_rebuild()
```

Light

Scene light lookup and per-type light properties.

Scene Lookup

API	Return	Notes
`scene.get_lights()`	`list[Light]`	All lights in current scene.
`scene.find_light(name)`	`Light | None`	Match by scene node name.
`scene.light_count`	`int`	Number of lights in the current scene.
`Sample.get_lights()`	`list[Light]`	Compatibility alias for `scene.get_lights()`.
`Sample.find_light(name)`	`Light | None`	Compatibility alias for `scene.find_light(name)`.
`Sample.set_environment_map(path)`	`None`	Override scene environment map source.

Base Class `Light`

Property	Type	Notes
`light_type`	implementation enum/int	Underlying Donut light type.
`name`	`str`	Scene node name. Read-only.
`color`	`(r, g, b)`	Writable.
`position`	`(x, y, z)`	Writable.
`direction`	`(x, y, z)`	Writable.

Derived classes expose extra properties depending on actual light type.

`DirectionalLight`

Property	Type	Notes
`irradiance`	`float`	Target illuminance, multiplied by `color`.
`angular_size`	`float`	Apparent angular size in degrees.

`SpotLight`

Property	Type
`intensity`	`float`
`radius`	`float`
`range`	`float`
`inner_angle`	`float`
`outer_angle`	`float`

`PointLight`

Property	Type
`intensity`	`float`
`radius`	`float`
`range`	`float`

`EnvironmentLight`

Property	Type
`radiance_scale`	`(r, g, b)`
`rotation`	float / implementation-specific value
`path`	`str`

For common environment tweaks, prefer `settings.environment_map` (see Settings) and `Sample.set_environment_map(path)`.

3DGS

3D Gaussian splat loading, scene graph nodes, and render settings.

Loading

API	Return	Notes
`Renderer.load_gaussian_splats(file_name, convert_rdf_to_donut=True)`	`bool`	Append a `.ply` 3DGS scene object (extension mode).
`Sample.load_gaussian_splats(file_name, convert_rdf_to_donut=True)`	`bool`	Append a 3DGS `.ply` node to the current scene.
Scene JSON `GaussianSplat` node	—	Declare splats in inline or file-based scene JSON. See Renderer → Inline / Builtin Scenes.

Read-only status on `Sample` / `Renderer.settings`:

Property	Type	Notes
`gaussian_splat_object_count`	`int`	Number of loaded 3DGS scene objects.
`gaussian_splat_count`	`int`	Total splat count across current 3DGS objects.
`gaussian_splat_file_name`	`str`	Single path or multi-object summary.

3DGS Enums

Enum	Values
GaussianSplatSortMode	GpuSort=0, StochasticSplats=1
GaussianSplatStorageFormat	Float32=0, Float16=1, Uint8=2
GaussianSplatFrustumCulling	Disabled=0, AtDistanceStage=1, AtRasterStage=2
GaussianSplatShadowMode	Disabled=0, Hard=1, Soft=2
GaussianSplatFTBSyncMode	Disabled=0, Interlock=1

Settings (settings.*)

3DGS data is scene-owned. Scene JSON can contain any number of GaussianSplat, GaussianSplats, or 3DGaussianSplat nodes. load_gaussian_splats(...) appends another GaussianSplat node to the current scene root. Switching scenes clears the old scene graph, including its 3DGS objects.

Rasterization runs over all enabled 3DGS scene objects. Emissive proxy sampling combines all enabled 3DGS objects into one world-space proxy list. Splat shadows currently use the first enabled 3DGS object as the primary shadow source.

gaussian_splat_translation, gaussian_splat_rotation_euler_deg, and gaussian_splat_object_scale apply only when Python appends a new 3DGS node through load_gaussian_splats(...).

Property	Type	Notes
enable_gaussian_splats	bool	Enables rendering for 3DGS scene objects.
gaussian_splat_depth_test	bool	Test against scene depth.
gaussian_splat_sorting_mode	int/GaussianSplatSortMode	GpuSort or StochasticSplats.
gaussian_splat_sh_format	int/GaussianSplatStorageFormat	SH payload storage format.
gaussian_splat_rgba_format	int/GaussianSplatStorageFormat	RGBA payload storage format.
gaussian_splat_use_aabbs	bool	Use AABB-based splat shadow acceleration data.
gaussian_splat_use_tlas_instances	bool	Use TLAS instances for splat shadow acceleration.
gaussian_splat_blas_compaction	bool	Enable BLAS compaction for splat shadow acceleration data.
gaussian_splat_mip_antialiasing	bool	Enable splat mip antialiasing path.
gaussian_splat_quantize_normals	bool	Quantize generated splat normals in the RTX path.
gaussian_splat_ftb_sync_mode	int/GaussianSplatFTBSyncMode	Front-to-back synchronization mode.
gaussian_splat_frustum_culling	int/GaussianSplatFrustumCulling	Frustum culling stage.
gaussian_splat_frustum_dilation	float	Culling frustum dilation.
gaussian_splat_screen_size_culling	bool	Enable screen-size splat culling.

Property	Type	Notes
`gaussian_splat_min_pixel_coverage`	`float`	Minimum pixel coverage for screen-size culling.
`gaussian_splat_depth_iso_threshold`	`float`	Depth/iso-surface threshold used by the splat path.
`gaussian_splat_fragment_shader_barycentric`	`bool`	Use fragment-shader barycentric path when supported.
`gaussian_splat_scale`	`float`	Projected footprint scale.
`gaussian_splat_alpha_scale`	`float`	Opacity multiplier.
`gaussian_splat_brightness`	`float`	Color multiplier.
`gaussian_splat_tint_color`	`(r, g, b)`	Multiplies the SH0/base color before brightness.
`gaussian_splat_as_emitter`	`bool`	Inject 3DGS emissive proxies into light sampling.
`gaussian_splat_emission_intensity`	`float`	Emissive proxy intensity multiplier.
`gaussian_splat_emission_max_proxy_count`	`int`	Emissive proxy budget.
`gaussian_splat_alpha_cull_threshold`	`float`	Cull low-alpha splats.
`gaussian_splat_translation`	`(x, y, z)`	Initial translation for newly attached Python 3DGS nodes.
`gaussian_splat_rotation_euler_deg`	`(x, y, z)`	Initial Euler rotation in degrees for newly attached Python 3DGS nodes.
`gaussian_splat_object_scale`	`(x, y, z)`	Initial non-uniform scale for newly attached Python 3DGS nodes.
`gaussian_splat_shadows`	`bool`	Enable splat shadow integration.
`gaussian_splat_hybrid_shadows`	`bool`	Alias for `gaussian_splat_shadows`.
`gaussian_splat_shadows_mode`	`int/GaussianSplatShadowMode`	Disabled, hard, or soft splat shadows.
`gaussian_splat_shadow_strength`	`float`	Shadow opacity/strength.
`gaussian_splat_shadow_soft_radius`	`float`	Soft shadow radius.
`gaussian_splat_shadow_soft_sample_count`	`int`	Soft shadow sample count.
`gaussian_splat_rtx_kernel_degree`	`int`	RTX splat kernel degree.
`gaussian_splat_rtx_adaptive_clamp`	`bool`	Enable adaptive RTX alpha clamp.

Property	Type	Notes
`gaussian_splat_rtx_alpha_clamp`	`float`	Manual RTX alpha clamp value.
`gaussian_splat_rtx_minimum_transmittance`	`float`	Minimum transmittance clamp for RTX splat tracing.
`gaussian_splat_rtx_trace_strategy`	`int`	RTX splat tracing strategy selector.
`gaussian_splat_rtx_particle_samples_per_pass`	`int`	RTX particle samples processed per pass.
`gaussian_splat_rtx_maximum_pass_count`	`int`	Maximum RTX splat trace pass count.
`gaussian_splat_rtx_particle_shadow_offset`	`float`	RTX particle shadow offset.
`gaussian_splat_rtx_particle_shadow_threshold`	`float`	RTX particle shadow threshold.
`gaussian_splat_rtx_colored_shadow_strength`	`float`	Strength for colored splat shadows.
`gaussian_splat_rtx_mesh_composite_threshold`	`float`	Mesh/splat composite threshold.
`gaussian_splat_rtx_depth_iso_threshold`	`float`	RTX depth/iso-surface threshold.
`gaussian_splat_object_count`	`int`	Read-only 3DGS scene object count.
`gaussian_splat_count`	`int`	Read-only total splat count.
`gaussian_splat_file_name`	`str`	Read-only single path or multi-object summary.

Settings

`Settings` mirrors the live ImGui UI state (`rtxpt.settings()` or `app.settings`). Most fields are writable and take effect on subsequent frames. For 3DGS-specific fields, see 3DGS.

General

Property	Type	Notes
`show_ui`	`bool`	Show/hide UI.
`enable_animations`	`bool`	Scene animation toggle.
`enable_vsync`	`bool`	VSync toggle.
`fps_limiter`	`float/int`	FPS limiter value.

Path Tracing Mode / Accumulation

Property	Type	Notes
`realtime_mode`	`bool`	`True` realtime, `False` reference.

Property	Type	Notes
`path_tracer_mode`	`int/PathTracerMode`	`Realtime=0`, `Reference=1`; changing it resets accumulation.
`realtime_samples_per_pixel`	`int`	SPP in realtime mode.
`accumulation_target`	`int`	Reference SPP target.
`reset_accumulation`	`bool`	Set `True` to reset accumulation.
`accumulation_aa`	`bool/int`	Accumulation AA toggle/setting.
`accumulation_prewarm_realtime_caches`	`bool`	Prewarm realtime caches before accumulation.

Path Tracer Knobs

Property	Type
`bounce_count`	`int`
`diffuse_bounce_count`	`int`
`enable_russian_roulette`	`bool`
`texture_lod_bias`	`float`

NEE / ReSTIR

Property	Type	Notes
`use_nee`	`bool`	Next event estimation.
`nee_type`	`int`	`0=uniform`, `1=power-based`, `2=NEE-AT`.
`nee_candidate_samples`	`int`	Candidate sample count.
`nee_full_samples`	`int`	Full sample count.
`nee_mis_type`	`int`	MIS mode.
`use_restir_di`	`bool`	ReSTIR direct illumination.
`use_restir_gi`	`bool`	ReSTIR global illumination.

Camera

Property	Type
`camera_aperture`	`float`
`camera_focal_distance`	`float`
`camera_move_speed`	`float`

Firefly Filters

Property	Type
`realtime_firefly_filter_enabled`	`bool`
`realtime_firefly_filter_threshold`	`float`
`reference_firefly_filter_enabled`	`bool`

Property	Type
reference_firefly_filter_threshold	float

Tone Mapping / Bloom

Property	Type
enable_tone_mapping	bool
enable_bloom	bool
bloom_intensity	float
bloom_radius	float

Realtime AA / DLSS / Reflex

Availability depends on build options and hardware support.

Property	Type	Notes
realtime_aa	int/RealtimeAA	0=Off, 1=TAA, 2=DLSS, 3=DLSS_RR.
dlss_mode	int/DLSSMode	DLSS quality preset.
dlss_lod_bias_use_override	bool	Override DLSS texture LOD bias.
dlss_lod_bias_override	float	LOD bias override.
dlss_always_use_extents	bool	Use DLSS extents mode.
dlss_fg_mode	int/DLSSFGMode	DLSS frame generation mode.
dlss_fg_multiplier	int	Frame generation multiplier.
dlss_fg_num_frames_to_generate	int	Current generated frame count.
dlss_fg_max_num_frames_to_generate	int	Max generated frame count.
dlss_rr_preset	int/DLSSRRPreset	DLSS Ray Reconstruction preset.
dlss_rr_micro_jitter	bool/float	DLSS-RR micro jitter setting.
dlss_rr_brightness_clamp_k	float	Brightness clamp factor.
disable_restirs_with_dlss_rr	bool	Disable ReSTIR features with DLSS-RR.
reflex_mode	int/ReflexMode	NVIDIA Reflex mode.
reflex_capped_fps	float/int	Reflex FPS cap.

Read-only support flags:

Property	Type
is_dlss_supported	bool
is_dlss_fg_supported	bool
is_dlss_rr_supported	bool
is_reflex_supported	bool

Denoisers

Realtime / NRD:

Property	Type	Notes
`standalone_denoiser`	`bool`	NRD denoiser in realtime mode; no effect with DLSS-RR.
`denoiser_radiance_clamp_k`	`float`	NRD radiance clamp.

Reference / OIDN:

Property	Type	Notes
`oidn_enabled`	`bool`	Run OIDN when accumulation completes.
`oidn_use_gpu`	`bool`	Use OIDN GPU device when available.
`oidn_passes`	`int/OidnPasses`	Auxiliary guide passes.
`oidn_prefilter`	`int/OidnPrefilter`	Guide prefilter quality.
`oidn_quality`	`int/OidnQuality`	Beauty filter quality.
`oidn_changed`	`bool`	Set true after edits; renderer clears it.
`oidn_apply()`	method	Marks OIDN parameters dirty.

Environment Map Runtime Parameters

`settings.environment_map` is an `EnvironmentMapParams` object:

Property	Type
`tint_color`	`(r, g, b)`
`intensity`	`float`
`rotation_xyz`	`(x, y, z)`
`enabled`	`bool`
`visible_to_camera`	`bool`
`hide_source`	`bool` inverse of `visible_to_camera`

Enums

All enums are arithmetic, so `int(enum_value)` works and enum values can be assigned to int-backed settings fields. 3DGS-specific enums are listed under 3DGS → 3DGS Enums.

Path Tracing & Realtime

`PathTracerMode`

Value	Int	Meaning
`rtxpt.PathTracerMode.Realtime`	`0`	Realtime path tracing mode.
`rtxpt.PathTracerMode.Reference`	`1`	Reference accumulation mode.

`RealtimeAA`

Value	Int	Meaning

Value	Int	Meaning
`rtxpt.RealtimeAA.Off`	`0`	No realtime AA/upscaler.
`rtxpt.RealtimeAA.TAA`	`1`	Temporal AA.
`rtxpt.RealtimeAA.DLSS`	`2`	DLSS Super Resolution.
`rtxpt.RealtimeAA.DLSS_RR`	`3`	DLSS Ray Reconstruction.

`DLSSMode`

Value	Int
`Off`	`0`
`MaxPerformance`	`1`
`Balanced`	`2`
`MaxQuality`	`3`
`UltraPerformance`	`4`
`UltraQuality`	`5`
`DLAA`	`6`

`DLSSFGMode`

Value	Int
`Off`	`0`
`On`	`1`
`Auto`	`2`

`DLSSRRPreset`

Value	Int
`Default`	`0`
`PresetA` … `PresetH`	`1` … `8`

`ReflexMode`

Value	Int
`Off`	`0`
`LowLatency`	`1`
`LowLatencyWithBoost`	`2`

OIDN

Enum	Values
`OidnPasses`	`ColorOnly=0`, `Albedo=1`, `AlbedoNormal=2`
`OidnPrefilter`	`None_=0`, `Fast=1`, `Accurate=2`

Enum	Values
`OidnQuality`	`Fast=0`, `Balanced=1`, `High=2`

Material

`TextureSlot`

Value	Meaning
`Base`	Base/diffuse texture.
`ORM` / `OcclusionRoughnessMetallic`	ORM texture, or spec-gloss texture when `use_specular_gloss=True`.
`Normal`	Normal texture.
`Emissive`	Emissive texture.
`Transmission`	Transmission texture.

Embedded Mode Notes

In embedded mode, scripts run inside `Rtxpt.exe`:

```
Rtxpt.exe --pythonScript Rtxpt/Python/Examples/example_basic.py
Rtxpt.exe --pythonExpr "import rtxpt; print(rtxpt.app().scene_name)"
```

Inside the app, the Python panel can run inline code. Typical script shape:

```
import rtxpt

app = rtxpt.app()
s = rtxpt.settings()

s.realtime_mode = True
s.realtime_aa = int(rtxpt.RealtimeAA.TAA)
app.reset_accumulation()
```

Do not create `rtxpt.Renderer` in embed mode; the running app already owns the renderer.

Extension Mode Notes

In extension mode, every `Renderer` owns a GPU device and scene. Use `close()` or a context manager so GPU resources are released promptly.

```
with rtxpt.Renderer(headless=True, scene="...") as r:
    ...
```

For windowed extension usage:

- `headless=False` opens a GLFW window.
- `Renderer.step()` must be called repeatedly to pump events and render frames.
- Clicking the window close button makes `step()` return `False`.
- Resize/maximize/minimize are handled by the underlying Donut `DeviceManager` during `step()`.

Existing Examples

File	Purpose
Rtxpt/Python/Examples/offline_render.py	Headless reference render and screenshot.
Rtxpt/Python/Examples/test_splat_interactive.py	Windowed or headless 3DGS rasterization test.
Rtxpt/Python/Examples/3dgs_example.py	Batch 3DGS Reference/OIDN and Realtime/DLSS-RR render test.
Rtxpt/Python/Examples/render_gs_colmap_views.py	Render 3DGS from COLMAP camera poses with full pinhole intrinsics and optional Mip antialiasing.
Rtxpt/Python/Examples/example_basic.py	Basic embedded scripting.
Rtxpt/Python/Examples/example_modes_dlss_oidn.py	Realtime/reference mode, DLSS, OIDN settings.
Rtxpt/Python/Examples/example_animate_lights.py	Per-frame light edits.

Introspection

The binding also exposes docstrings through nanobind:

```
import rtxpt
help(rtxpt)
help(rtxpt.Renderer)
help(rtxpt.Sample)
help(rtxpt.Settings)
```